

UNIVERSIDAD DEL VALLE DE GUATEMALA

Facultad de Ingeniería

Data Science – LYNETTE GARCIA PEREZ

Sección 10



Laboratorio 3

Javier Chen, 22153

Gustavo Cruz, 22779

Introducción:

El objetivo de este laboratorio fue desarrollar e implementar modelos de aprendizaje profundo capaces de reconocer dígitos manuscritos utilizando el conjunto de datos PolyMNIST, una versión compleja del clásico MNIST que introduce variabilidad en el fondo y en la posición de los dígitos. A través de la construcción y evaluación de modelos como redes neuronales, se buscó analizar su desempeño, comparar su efectividad y determinar qué modelo es más robusto frente a las condiciones visuales adversas presentes en el dataset.

Descripción de los datos que se utilizaron:

PolyMNIST es una extensión del dataset MNIST que introduce un mayor nivel de complejidad visual. El conjunto cuenta con cinco modalidades distintas, en las cuales cada imagen combina un dígito manuscrito con un fondo generado a partir de fragmentos aleatorios extraídos de una imagen mayor. Esta configuración tiene como resultado:

- Información de fondo única para cada modalidad, lo que introduce ruido visual y variabilidad de contexto.
- Información compartida: el dígito manuscrito, que es el mismo entre modalidades, pero aparece en diferentes posiciones y contextos.

Una característica adicional que incrementa el desafío del problema es que los dígitos están trasladados aleatoriamente dentro del recorte, lo que obliga a los modelos a aprender a detectar el dígito en diferentes ubicaciones espaciales.

Análisis Exploratorio:

Estructura del dataset

TRAIN:

- m0: 60000 imágenes
- m1: 60000 imágenes
- m2: 60000 imágenes
- m3: 60000 imágenes
- m4: 60000 imágenes

TEST:

- m0: 10000 imágenes
- m1: 10000 imágenes
- m2: 10000 imágenes
- m3: 10000 imágenes
- m4: 10000 imágenes

Dimensiones únicas por modalidad:

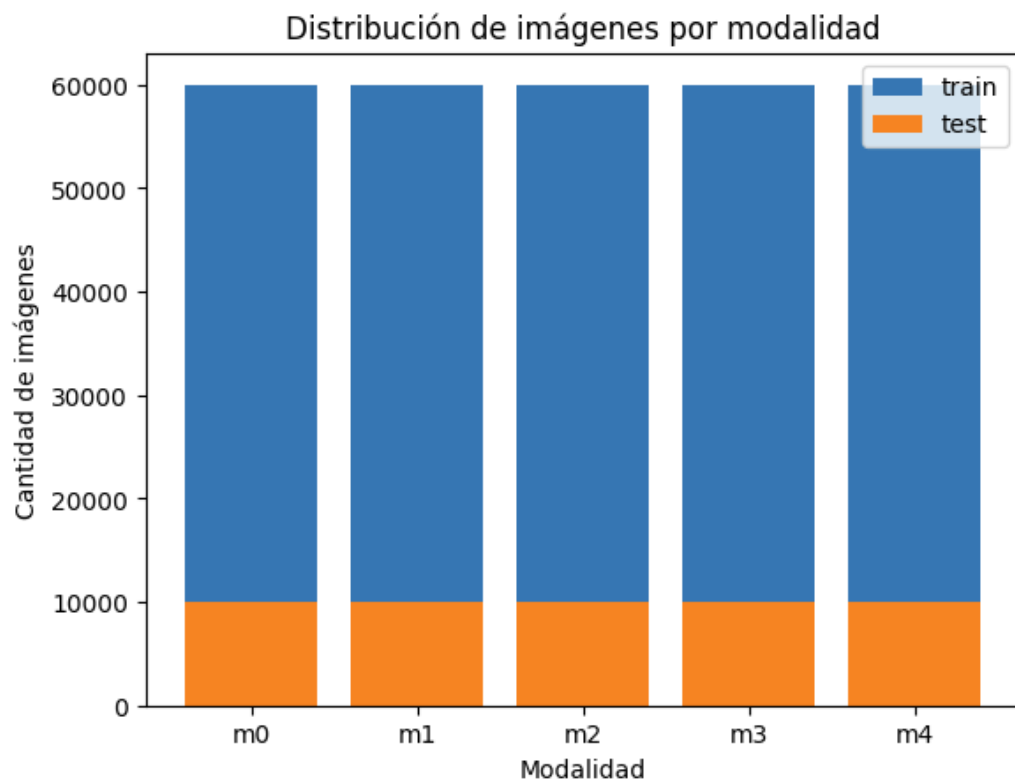
m0: {(28, 28)}

m1: {(28, 28)}

m2: {(28, 28)}

m3: {(28, 28)}

m4: {(28, 28)}



Ejemplos de las imágenes del dataset:



Ejemplos de cada modalidad (m0 a m4)



Resultados y Evaluación de Modelos:

Modelos sin Transformaciones de Imagen

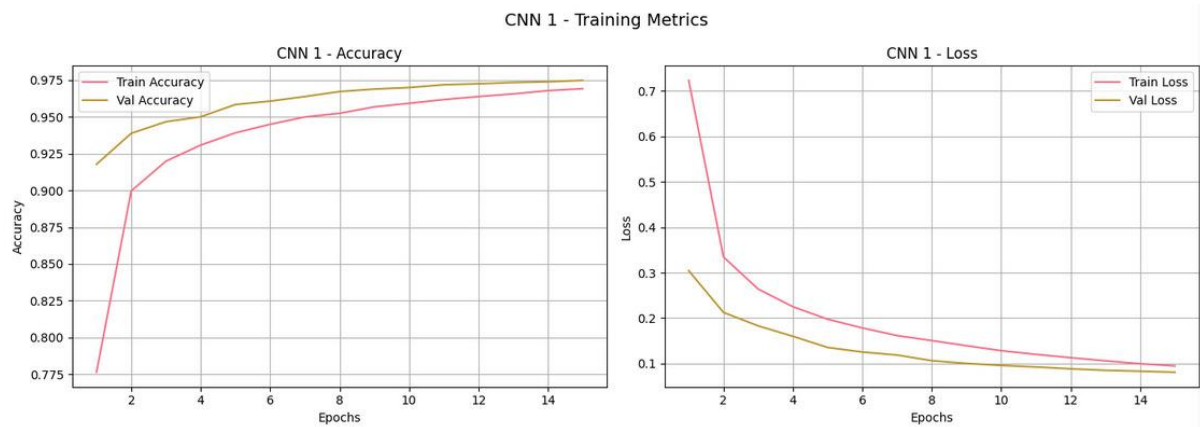
Cambio aplicado: la imagen se pasó a una escala de grises

CNN 1 – Modelo Convolutivo Básico:

- Capas: 2 convolucionales + MaxPooling + Dense final
- Épocas entrenadas: 15
- Tasa de aprendizaje inicial: 0.0005

Resultados finales:

- Accuracy entrenamiento: 96.91%
- Accuracy validación: 97.47%
- Loss validación: 0.0809



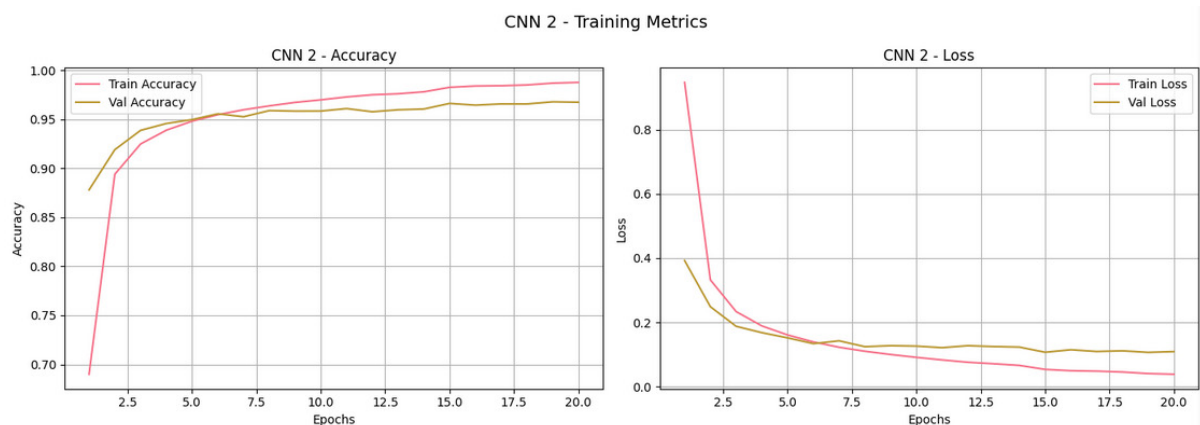
Este modelo tuvo un aprendizaje progresivo y estable, sin señales de sobreajuste. Logró una precisión bastante alta para un modelo de arquitectura básica

CNN 2 – Modelo Convolutacional Optimizado

- Capas: más profundas con Dropout y ReduceLROnPlateau
- Épocas entrenadas: 20
- Tasa de aprendizaje inicial: 0.0003

Resultados finales:

- Accuracy entrenamiento: 98.76%
- Accuracy validación: 96.78%
- Loss validación: 0.1096



Aunque tiene mayor profundidad y precisión en entrenamiento, su rendimiento en validación fue ligeramente inferior al CNN 1.

MLP – Red Neuronal Multicapa (Simple)

- Capas: Flatten → Dense(128) → ReLU → Dense(10) → Softmax
- Épocas entrenadas: 15
- Tasa de aprendizaje: 0.001

Resultados finales:

- Accuracy entrenamiento: 77.07%
- Accuracy validación: 80.08%
- Loss validación: 0.6115



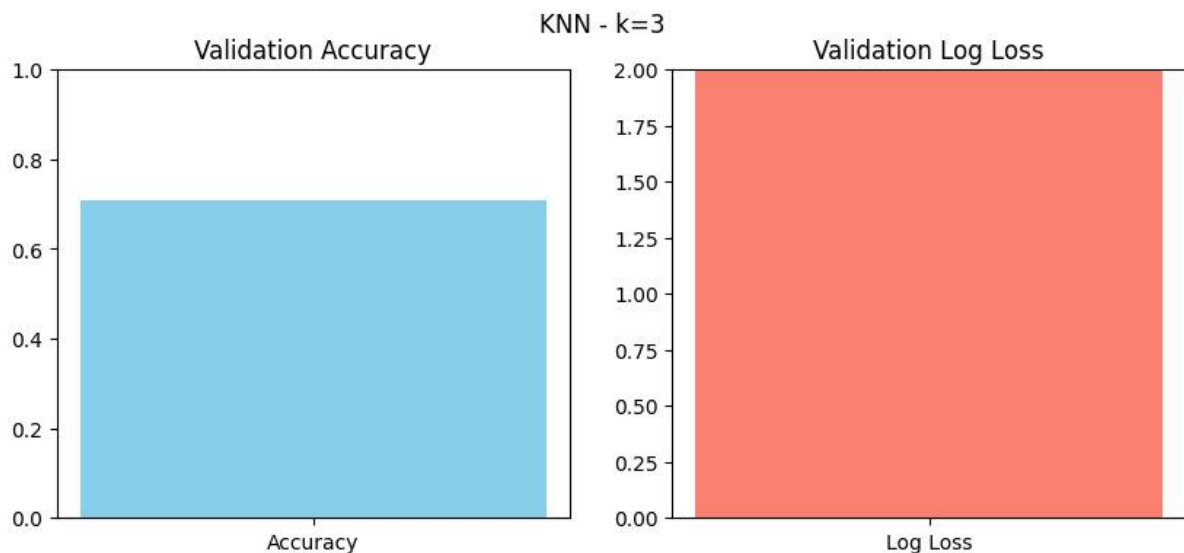
El modelo simple obtuvo resultados modestos. Si bien aprendió gradualmente, tiene una capacidad limitada para captar patrones espaciales, lo cual explica su menor rendimiento.

Modelo basado en KNN (k=3)

Se implementó un modelo K-Nearest Neighbors (KNN) con $k=3$, utilizando las imágenes planas.

Resultados en el conjunto de validación:

- Accuracy: ~70%
- Log Loss: ~2.0



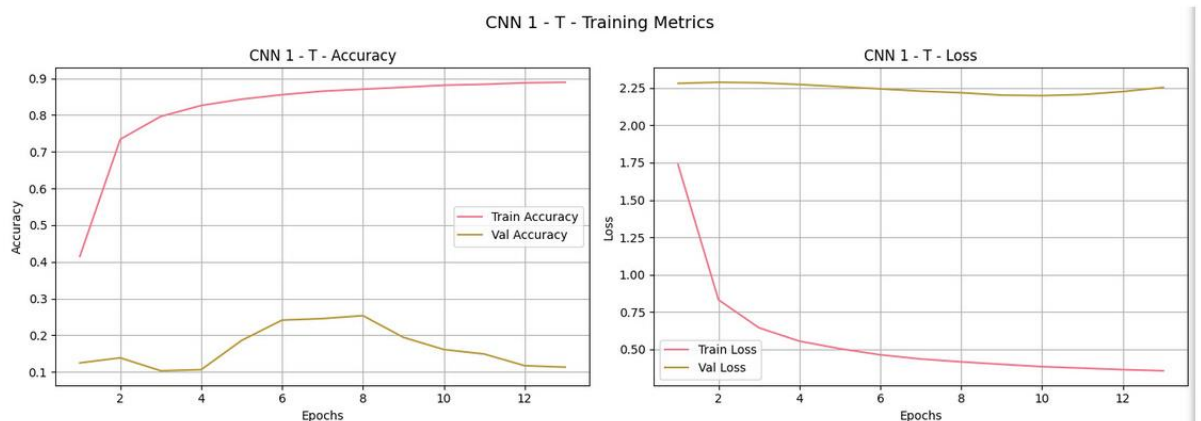
Aunque la precisión es aceptable para un modelo simple, el alto log loss indica baja confianza en sus predicciones. Esto sugiere que KNN no es competitivo frente a redes neuronales para este problema, especialmente por la variabilidad de los fondos en el dataset.

Modelos con Transformaciones de Imagen:

Transformaciones aplicadas: rotaciones.

CNN 1 con Augmentation

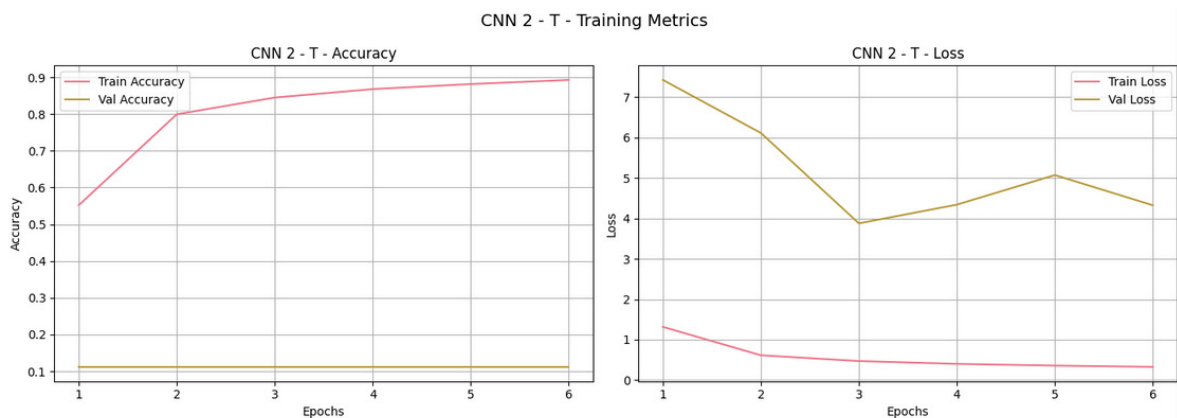
- Accuracy entrenamiento: 88.95%
- Accuracy validación: 25.34% (mejor valor)
- Loss validación: 2.2191



A pesar de mejorar en entrenamiento, no generalizó bien en validación. El modelo claramente no se adaptó a las imágenes rotadas, probablemente porque la transformación causó que los dígitos fueran difíciles de distinguir.

CNN 2 con Augmentation

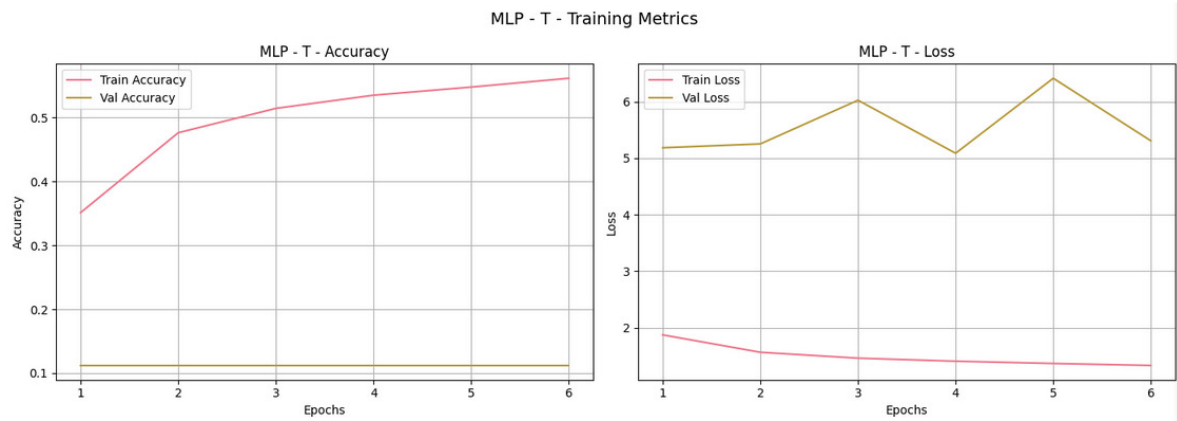
- Accuracy entrenamiento: 89.29%
- Accuracy validación: 11.21%
- Loss validación: 4.3261



El modelo no logró aprender con efectividad. Todas las épocas mantuvieron la misma precisión en validación 11.21%. Se puede concluir que hubo un problema severo de sobreajuste que el modelo fue incapaz de adaptarse al ruido artificial.

MLP con Augmentation

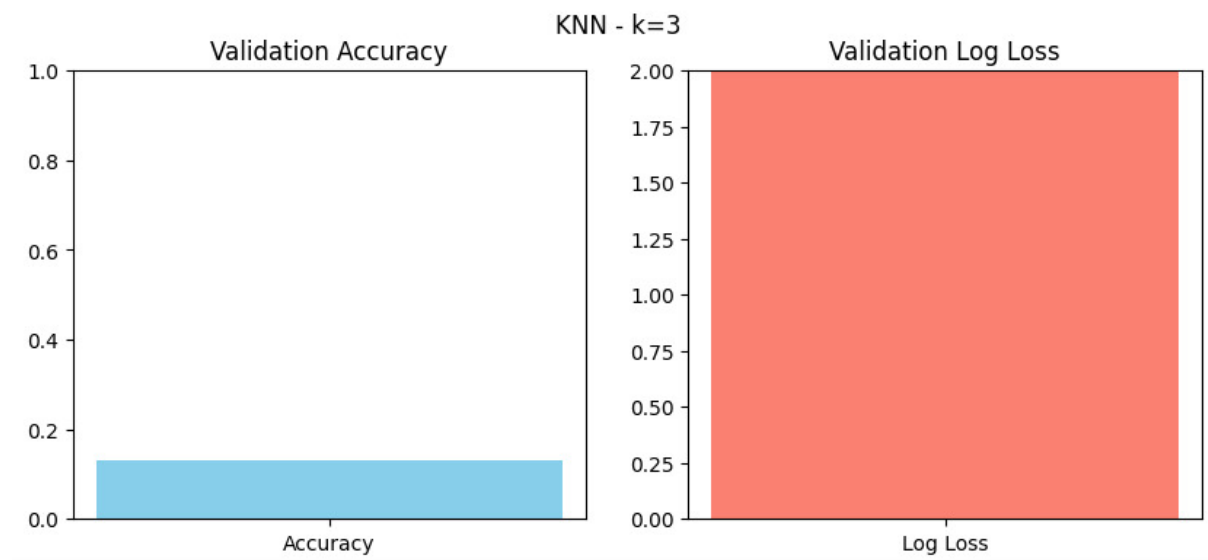
- Accuracy entrenamiento: 56.17%
- Accuracy validación: 11.21%
- Loss validación: 5.35



Este modelo también falló completamente en validar con datos aumentados. Las imágenes transformadas afectaron su capacidad de interpretación, reforzando que los modelos simples son especialmente vulnerables a variaciones en la imagen.

Modelo basado en KNN con datos aumentados

Se volvió a aplicar el modelo KNN ($k=3$), esta vez utilizando el conjunto de imágenes transformadas con image augmentation.



Resultados en validación:

- Accuracy: ~13%
- Log Loss: ~2.0

Este modelo presentó un rendimiento muy bajo, incluso por debajo del azar. La baja precisión y el alto log loss indican que KNN no logra adaptarse a la variabilidad

introducida por las transformaciones, confirmando que este algoritmo es altamente sensible a cambios espaciales y visuales en las imágenes.

Comparación de los algoritmos:

Modelo	Accuracy Entrenamiento	Accuracy Validación	Log Loss Validación	Observaciones
CNN 1	96.91 %	97.47%	0.0809	Mejor resultado, buena generalización
CNN 2	98.76 %	96.78%	0.1096	Ligero sobreajuste, aún muy buen rendimiento
MLP	77.07 %	80.08%	0.6115	Limitado por su arquitectura simple
KNN (k=3)	—	~70%	~2.0	Rendimiento decente pero pobre en “confianza”
CNN 1 (Transformación)	88.95 %	25.34%	2.2	Alto sobreajuste, mal aprendizaje con augmentación
CNN 2 (Transformación)	89.29 %	11.21%	4.3	Fallo total en generalizar con augmentación
MLP (Transformación)	56.17 %	11.21%	5.3	Inestable, incapaz de adaptarse
KNN (k=3) (Transformación)	—	13%	~2.0	Muy sensible a transformaciones visuales

- El modelo CNN 1 sin transformaciones fue el más efectivo y consistente, alcanzando un 97.47% de precisión en validación, con una pérdida logarítmica baja, y sin signos evidentes de sobreajuste.
- CNN 2, a pesar de su complejidad, no superó al CNN 1, y mostró señales leves de sobreajuste. Sin embargo, sigue siendo altamente competitivo.
- El modelo MLP, al carecer de capas convolucionales, tuvo un rendimiento moderado, adecuado para comparación, pero claramente inferior a las CNNs.

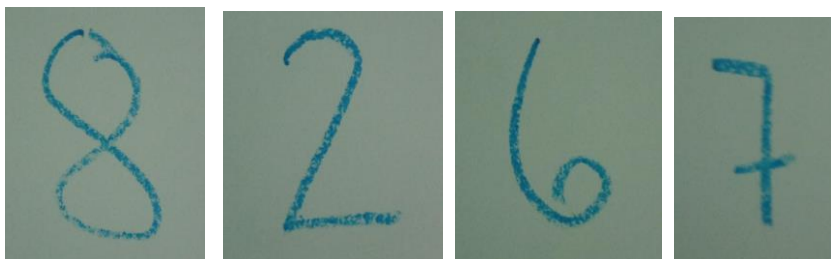
- El KNN (k=3) mostró una precisión aceptable sin augmentación, pero muy baja confianza, como lo refleja su log loss elevado. No es adecuado para tareas de clasificación con imágenes ruidosas o complejas.
- Todos los modelos evaluados con augmentación de imagen (CNNs, MLP y KNN) fallaron rotundamente. La precisión cayó drásticamente y las pérdidas aumentaron. Esto indica que las transformaciones aplicadas fueron posiblemente demasiado agresivas o no adecuadas para el dataset PolyMNIST, el cual ya contiene variaciones complejas de fondo.

Conclusión:

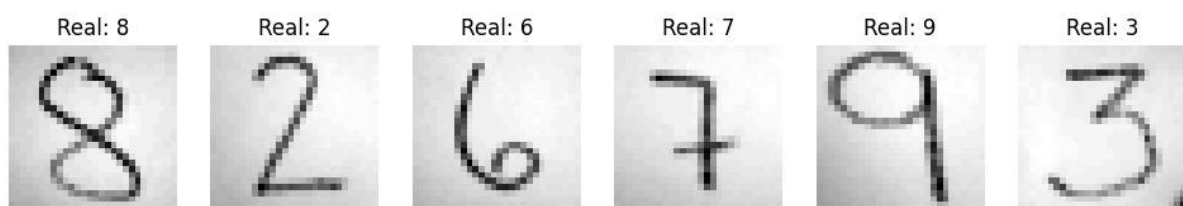
El mejor enfoque para el reconocimiento de caracteres manuscritos en el dataset PolyMNIST es el uso de modelos CNN clásicos y bien entrenados con imágenes sin transformación. La información visual del fondo ya aporta suficiente variabilidad, y aplicar transformaciones adicionales parece perjudicar el proceso de aprendizaje.

Modelos con imágenes hechas a mano:

Imágenes usadas:



Imágenes hechas a mano



Modelo	Predicciones	Reales	Accuracy
--------	--------------	--------	----------

CNN 1	[8, 1, 6, 7, 4, 1]	[8, 2, 6, 7, 9, 3]	0.50
CNN 2	[2, 2, 6, 7, 7, 1]	[8, 2, 6, 7, 9, 3]	0.50
MLP	[8, 1, 0, 1, 6, 1]	[8, 2, 6, 7, 9, 3]	0.17
CNN 1 (Transformación)	[1, 1, 1, 1, 1, 1]	[8, 2, 6, 7, 9, 3]	0.00
CNN 2 (Transformación)	[1, 1, 1, 1, 1, 1]	[8, 2, 6, 7, 9, 3]	0.00
MLP (Transformación)	[1, 1, 1, 1, 1, 1]	[8, 2, 6, 7, 9, 3]	0.00
KNN	[1, 1, 1, 2, 1, 1]	[8, 2, 6, 7, 9, 3]	0.00
KNN (Transformación)	[1, 1, 1, 1, 1, 1]	[8, 2, 6, 7, 9, 3]	0.00

Resultados:

- Los mejores modelos en validación no necesariamente son robustos con datos reales no vistos.
- CNN 1 sin transformación fue el más balanceado entre rendimiento y generalización.
- Las técnicas de image augmentation deben ser cuidadosamente diseñadas, ya que, en este experimento, degradaron fuertemente el desempeño.
- KNN y MLP no son adecuados para este tipo de tarea con datos complejos y ruidosos.

Este experimento resalta la importancia de validar los modelos con escenarios reales y no depender únicamente del rendimiento en datasets limpios o controlados.

Conclusiones:

El experimento demostró que las redes convolucionales (CNN) son las más efectivas para clasificar dígitos manuscritos en contextos visuales complejos como los del dataset PolyMNIST.

En cambio, modelos más simples como **MLP** y **KNN** tuvieron un rendimiento significativamente inferior, especialmente al enfrentarse a ruido visual o transformaciones. Además, aplicar **image augmentation** resultó contraproducente en todos los modelos, degradando fuertemente su desempeño debido a la complejidad visual ya presente en el dataset.

Finalmente, se evidenció que un buen rendimiento en validación no garantiza robustez ante datos reales. Esto resalta la importancia de probar los modelos en escenarios menos controlados.

Referencias:

Github: https://github.com/JavierC22153/Laboratorios_DataScience/tree/Lab3